

目 录

前言	xvii
第1阶段 自动微分	1
步骤1 作为“箱子”的变量	3
1.1 什么是变量	3
1.2 实现 Variable 类	4
1.3 (补充) NumPy 的多维数组	6
步骤2 创建变量的函数	8
2.1 什么是函数	8
2.2 Function 类的实现	9
2.3 使用 Function 类	10
步骤3 函数的连续调用	13
3.1 Exp 函数的实现	13
3.2 函数的连续调用	14
步骤4 数值微分	16
4.1 什么是导数	16
4.2 数值微分的实现	17
4.3 复合函数的导数	20
4.4 数值微分存在的问题	21

步骤5	反向传播的理论知识	22
5.1	链式法则	22
5.2	反向传播的推导	23
5.3	用计算图表示	25
步骤6	手动进行反向传播	27
6.1	Variable类的功能扩展	27
6.2	Function类的功能扩展	28
6.3	Square类和Exp类的功能扩展	28
6.4	反向传播的实现	29
步骤7	反向传播的自动化	32
7.1	为反向传播的自动化创造条件	33
7.2	尝试反向传播	36
7.3	增加backward方法	38
步骤8	从递归到循环	40
8.1	现在的Variable类	40
8.2	使用循环实现	41
8.3	代码验证	42
步骤9	让函数更易用	43
9.1	作为Python函数使用	43
9.2	简化backward方法	45
9.3	只支持ndarray	46
步骤10	测试	50
10.1	Python的单元测试	50
10.2	square函数反向传播的测试	52
10.3	通过梯度检验来自动测试	53
10.4	测试小结	54

前言

真正的发现之旅，不在于寻找新的风景，
而在于拥有新的眼光。

——马塞尔·普鲁斯特(法国作家，1871—1922)

如今，深度学习正在推动各个领域的创新。自动驾驶技术、疾病自动诊断技术、高精度机器翻译技术、先进的机器人控制技术……这些仿佛在虚拟世界中才会出现的技术近年来正在成为现实，而且在实际生活中得到了应用。令人惊讶的是，这些技术大多因为深度学习才得以实现(或者正在实现)。可以说我们生活在一个由深度学习改变世界的时代。

这股深度学习的热潮催生出众多深度学习框架，包括PyTorch、Chainer^①、TensorFlow和Caffe等。深度学习领域有很多不同的框架，这些框架每天都在相互竞争中发展。得益于此，全世界的研究人员和工程师可以高效地解决很多问题。可以说深度学习框架是支持和推进深度学习技术发展的不可或缺的动力。

阅读本书的读者也许用过深度学习框架。如今深度学习的学习资料非常丰富，运行环境也已完善，因此，编写深度学习代码也变得非常容易。得益于这些框架，我们只用几十行(甚至几行)代码就可以实现复杂的技术。

^① Chainer是作者所在的Preferred Networks公司推出的开源机器学习库，在日本有较大影响。Chainer首创的Define-by-Run机制能够以直观、灵活的方式构建复杂的神经网络，获得了研究人员和开发者社区的支持。遗憾的是，2019年12月Preferred Networks公司正式宣布停止Chainer新版本的开发，而转向PyTorch。——译者注

那么，这些被许多人用在许多地方的框架实际上是如何工作的呢？它们采用了什么样的技术，又有什么样的思想作为支撑呢？让我们带着这些问题，开启一段新的旅程吧！

只有创建，才能看见

深度学习的框架中包含着很多惊人的技术和有趣的机制。本书旨在解读它们，帮助读者正确地理解这些技术，让读者感受到这些技术的有趣之处。为了实现这一目标，本书采取“从零开始创建”的做法，也就是从零开始边开发边思考，让大家在运行程序的过程中加深理解。通过这样的体验，读者将了解到深度学习框架的本质。

在创建框架的过程中，读者会学到很多东西，会体验到“我明白了，原来可以用这个技术！”“原来这个想法可以这样实现！”之类的瞬间，这些体验是仅仅使用现有工具所无法获得的。有些东西，只有创建，才能理解；有些东西，只有创建，才能被看见。

比如，有人可能认为深度学习框架只不过是一个拼凑了层和函数等组件的库，其实深度学习框架包含的东西远不止于此。可以说框架是一种编程语言，具体来说，它是一种具有求导功能的编程语言（最近也有人称它为“可微分编程语言”）。阅读本书，体验从零开始创建的过程，你就能体会到这一点。

本书的原创框架

深度学习框架在黎明期彼此差异很大，而现代深度学习框架已经进入成熟期，PyTorch、Chainer 和 TensorFlow 等被广泛使用的框架实际上都在朝着同一个方向发展（当然，它们有各自的特点，有不同的用户接口，但它们的设计思想正趋于一致）。本书基于这些共性，设计了一个重视教学的极简框架。因为是从零开始创建的，所以笔者将这个框架命名为 DeZero，并为它制作了图 1 所示的标识。



图1 DeZero 的标识

DeZero 是本书原创的框架。它的实现以 Chainer 为基础，同时引入了 PyTorch 的设计。下面笔者来详细介绍一下它的特点。

1. 极简主义

DeZero 是以简单易懂为第一设计原则的框架。在设计方面尽量减少了外部库的使用，内部代码也压缩到了最简。因此，读者不用花费很多时间就能理解 DeZero 的全部代码。

2. 纯 Python

许多深度学习框架使用多种编程语言（如 Python 和 C++ 等）来实现，而 DeZero 只用 Python 来实现。因此，只要懂 Python，就可以毫无障碍地阅读 DeZero 的代码。由于该框架只使用 Python 来实现，所以我们可以轻松地在智能手机上，或者使用 Google Colaboratory 等服务在云端运行它。

3. 具备现代深度学习框架的功能

PyTorch、Chainer 和 TensorFlow 等现代深度学习框架有许多相同的功能，其中一个重要的功能是 Define-by-Run。Define-by-Run 是在进行深度学习计算时在计算之间建立“连接”的机制（正文中将详细介绍这个特性）。本书创建的 DeZero 框架就是一个 Define-by-Run 风格的框架，它采用了许多与现代深度学习框架相同的设计。



本系列的前作《深度学习入门：基于Python的理论与实现》《深度学习进阶：自然语言处理》带领大家从零开始实现深度学习，并介绍了它的工作原理。不过，这两本书在实现时以简单为先，计算之间的“连接”都是手动设置的。而真正的框架会自动实现连接，这就需要用到 Define-by-Run。本书通过从零开始构建 DeZero 来让读者了解它的工作原理。在阅读本书之前，读者不需要事先阅读本系列的前两部作品。

增量开发

DeZero 虽然是一个小框架，但它的内容足够复杂。为了化繁为简，本书将创建 DeZero 的工程划分为多个小步骤。具体来说，就是依次完成 60 个步骤，一点点地创建 DeZero，本书的目录也是按步骤划分的。

比如步骤 1 是创建 DeZero 的变量，完成这个步骤只需 3 行代码。步骤 2 是给函数编写代码。各步骤的内容都是独立完成的，代码可以实际运行。在本书中，我们会通过这种增量开发的方式来逐步创建 DeZero，一边运行代码一边加深理解。

此外，通过本书得到的经验也可以运用在软件开发中。体验从头开始创建一个复杂系统的过程是学习软件开发的好方法。考虑到这一点，本书也使用了一些篇幅来介绍软件开发方法的相关内容。

本书的路线图

如前所述，本书由 60 个步骤构成，这 60 个步骤又可分为如图 2 所示的 5 个阶段。下面笔者简单介绍一下每个阶段的内容。

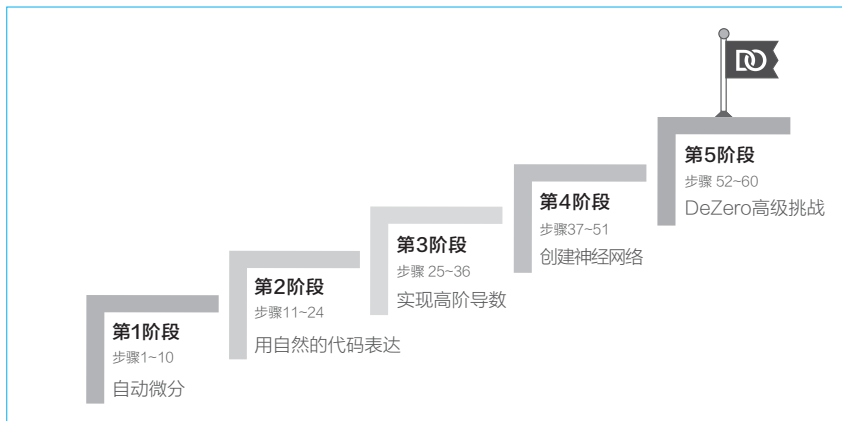


图2 本书结构

- 在**第1阶段**，我们会为创建 DeZero 打好基础。在这个阶段，我们只处理简单的问题，用最短的时间实现自动微分的功能（在阅读本书的过程中，读者会逐渐理解自动微分的意思）。
- 在**第2阶段**，我们将扩展 DeZero，使其能以更自然的代码来表达。在第2阶段结束时，我们应该就能使用平常用到的 Python 语法，比如 if 语句和 for 语句等来编写代码。
- 在**第3阶段**，我们将继续扩展 DeZero，使它可以计算二阶导数。要做到这一点，需要使 DeZero 实现“反向传播的反向传播”。通过了解其工作原理，我们会发现 DeZero 和现代深度学习框架中蕴含的新的可能性。
- 在**第4阶段**，我们会让 DeZero 实现神经网络的功能。这样一来，我们就能用 DeZero 轻松地构建神经网络。
- 最后在**第5阶段**，我们将增加对 GPU 的支持、模型的保存和恢复等深度学习必备的功能，还会研究 CNN 和 RNN 等更先进的模型。在深度学习的应用方面，这些都是很重要的问题，但这些问题都不好解决。不过，在这个阶段我们会了解到，（具备 Define-by-Run 能力的）DeZero 可以用简单的代码来解决这些问题。



本书最后完成的DeZero已经被发布到了PyPI(Python Package Index)中。PyPI是Python的软件包仓库。大家可以在命令行使用`pip install dezero`命令安装DeZero。当然,也可以在本书创建的DeZero的基础上开发自己的框架,并在网上公开。

踏上创建 DeZero 之旅

以上就是本书的大致内容。总结一下,本书介绍了从零开始创建原创框架DeZero的过程。DeZero是一个小而强大的框架,我们将通过60个步骤来完成它。在这一过程中,读者会加深对PyTorch、Chainer和TensorFlow等现代深度学习框架的理解。

最后再次强调,本书的目的不仅仅是创建原创框架DeZero,更重要的是通过创建DeZero的旅程,让大家拥有观察现代深度学习框架的“新的视角”。拥有了新的视角,我们就能更广泛、更深入地探索深度学习领域。发现这个新的视角的过程,才是本书真正的价值所在。

好了,准备工作完成了,下面就让我们开启创建DeZero之旅吧!

必要的软件

本书使用的Python版本及外部库如下所示。

- Python 3
- NumPy
- Matplotlib
- CuPy(可选)
- Pillow(可选)

DeZero还提供了能在NVIDIA的GPU上运行的功能作为可选项。要实现这个功能,我们需要用到名为CuPy的外部库。同样,我们还需要选配

Pillow 这个图像处理库。除 Python 之外，本书还使用了以下软件。

- Graphviz

笔者将在[步骤 25](#)介绍如何安装 Graphviz。

文件构成

本书使用的代码可以从以下网址获取。

ituring.cn/book/2863

各文件夹的内容如表 1 所示。

表 1 文件夹的内容

文件夹名	说明
dezero	DeZero 的源代码
examples	使用 DeZero 开发的示例
steps	各步骤的代码文件 (step01.py ~ step60.py)
tests	DeZero 的单元测试

steps 文件夹中的 step01.py、step02.py 等文件与本书各步骤中创建的文件一一对应。我们可以通过以下 Python 命令运行这些文件 (可以在任何目录下运行 Python 命令)。

```
$ python steps/step01.py
$ python steps/step02.py

$ cd steps
$ python step31.py
```


第1阶段

自动微分

导数广泛应用在现代科学技术的各个领域，尤其在包括深度学习在内的机器学习的各个领域，导数起着核心作用。从某种意义上来说，深度学习框架就是计算导数的工具。因此，本书的主题自然包括导数。换言之，如何利用计算机求导是本书的一个重要知识点。

马上要进入的第1阶段共包括10个步骤。在这个阶段，我们将创建自动微分的机制。这里所说的自动微分指的是由计算机(而不是人)来计算导数。具体来说，就是指在对某个计算(函数)编码后，由计算机自动求出该计算的导数的机制。

在这个阶段，我们将创建代表变量和函数的两个类(`Variable`类和`Function`类)。让人惊讶的是，有了这两个类，我们就为自动微分打好了基础。在第1阶段结束时，对于简单的计算(函数)，DeZero应该可以自动求出它的导数了。下面就进入创建DeZero的首个阶段吧！

步骤1

作为“箱子”的变量

本书的第1个步骤是创建DeZero的组成元素——变量。变量是DeZero最重要的组成部分。在这个步骤中，我们将思考变量是如何工作的，并实现变量的功能。

1.1 什么是变量

首先来看一下什么是变量。打开编程入门类图书，你会发现这些图书大多用类似于图1-1的示意图来解释变量。

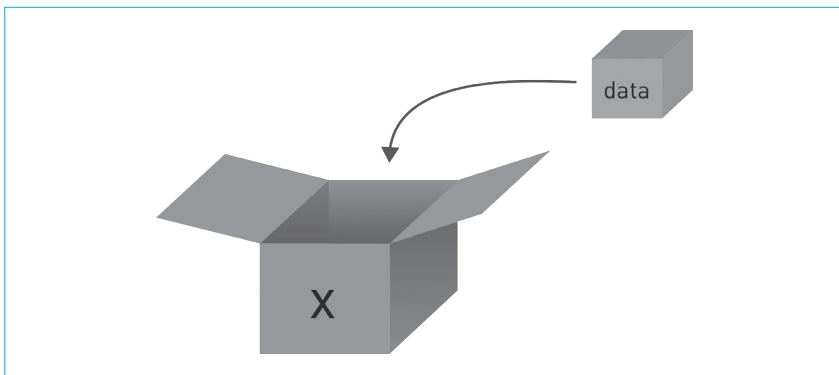


图1-1 变量的示意图

然后这些图书会告诉你，“箱子”就是变量，里面可以存放数据。这种把变量比作“箱子”的说法（在一定程度上）很好地解释了变量的特点。变量的要点可以总结如下。

- “箱子”和数据是不同的东西
- “箱子”里可以存放数据（=赋值）
- 朝“箱子”里看一看就能知道数据是什么（=引用）

下面来实现DeZero的变量，使它具有示意图中“箱子”的特点。

1.2 实现Variable类

变量的英语是variable。这里我们把DeZero的变量以Variable类的形式实现。补充一句，类名首字母大写是Python的一条常见的编码规则。这条规则记载在Python的PEP8编码规范中。

下面编写代码，让Variable类具备“箱子”的功能。实现这个功能的最简代码如下所示。

steps/step01.py

```
class Variable:
    def __init__(self, data):
        self.data = data
```

上面的代码所做的是在初始化时，将传来的参数设置为实例变量data。虽然这段代码很简单，但它可以使Variable类作为“箱子”使用，因为实际的数据被保存在Variable的data里。结合下面的使用示例，我们能更清晰地看出这一点。

steps/step01.py

```
import numpy as np

data = np.array(1.0)
x = Variable(data)
print(x.data)
```

运行结果

1.0

这个例子使用了NumPy的多维数组来保存放入“箱子”里的数据。例子中的x是一个Variable实例，实际的数据在x里面。换言之，x不是数据，而是存放数据的实体，也就是一个存放数据的“箱子”。



本书在展示代码时，会在代码的右上角标明它出自哪个文件。在上面的示例代码中，代码的右上角写着steps/step01.py，这表示你可以在本书的随书下载文件^①中的step/step01.py中找到这段代码。有些示例代码的右上角没有记载文件名，这种情况意味着随书下载文件里没有对应的代码文件。

机器学习系统使用多维数组作为底层数据结构。因此，DeZero的Variable类要设计成只能处理NumPy多维数组的形式。NumPy多维数组的类是numpy.ndarray(np.ndarray)。如前面的代码所示，我们可以使用np.array函数来创建它的实例。另外，本书后面在提到numpy.ndarray实例时，都将其简称为ndarray实例。

接下来为前面代码中的x赋新的数据。代码编写如下。

steps/step01.py

```
x.data = np.array(2.0)
print(x.data)
```

运行结果

2.0

^① 随书下载文件通过前言中提到的网址获取。——编者注

如这段代码所示，只要按照 `x.data = ...` 的方式编写，`x` 就会被赋上新的数据。这样 `Variable` 类就可以作为一个“箱子”使用了。

以上就是步骤1实现的所有代码。虽然现在 `Variable` 类只有3行代码，但我们会在此基础上将 `DeZero` 打成一个现代框架。

1.3 （补充）NumPy 的多维数组

最后简单介绍一下 NumPy 的多维数组。多维数组是一种数值等元素按照一定规则排列的数据结构。元素的排列顺序是有方向的，这个方向叫作维或轴。图 1-2 是一个多维数组的示例。

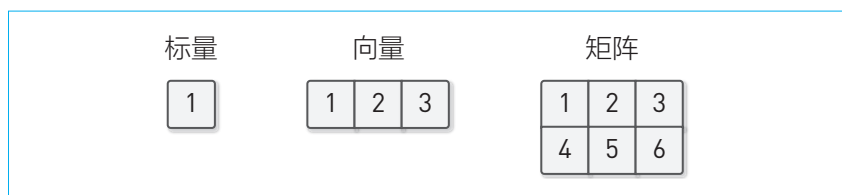


图 1-2 多维数组的示例

图 1-2 中从左到右依次为零维矩阵、一维矩阵、二维矩阵，它们分别叫作标量、向量和矩阵。标量就是简单的一个数。数沿着一个轴排列的是向量，数沿着两个轴排列的是矩阵。



多维数组也叫张量。图 1-2 中从左到右也可以分别叫作零阶张量、一阶张量和二阶张量。

NumPy 的 `ndarray` 实例中有一个实例变量叫 `ndim`，`ndim` 是 `number of dimensions` 的缩写，表示多维数组的“维度”。下面我们实际用一用它。


```
>>> import numpy as np
>>> x = np.array(1)
>>> x.ndim
0

>>> x = np.array([1, 2, 3])
>>> x.ndim
1

>>> x = np.array([[1, 2, 3],
...               [4, 5, 6]])
>>> x.ndim
2
```

上面的代码是启动了Python解释器的交互模式后，在交互模式下运行的（在交互模式下，代码前会出现>>>符号）。如上所述，通过实例变量`ndim`，我们可以知道数组的维度。



只有在处理向量的时候，我们才需要注意“维度”这个词。比如有一个向量`np.array([1, 2, 3])`，由于它有3个元素，所以也叫三维向量。向量的维度指的是向量中元素的数量。而在提到三维数组时，数组的维度是指轴（而不是元素）的数量是3。

如上所述，`ndarray`实例可以用来创建标量、向量、矩阵，甚至更高阶的张量。不过本书暂时只涉及标量的计算。后面（从[步骤37](#)开始）我们将扩展DeZero，使其支持向量和矩阵的处理。

步骤2

创建变量的函数

在上一步骤中，`Variable`类可以作为一个“箱子”使用了。不过，现在的它还只是一个“普通箱子”，我们需要一个机制把这个“普通箱子”变为“魔法箱子”。要实现这一点，关键就在于函数。本步骤的主题是函数。

2.1 什么是函数

什么是函数？函数是“定义一个变量与另一个变量之间的对应关系的规则”。这个说法有些生硬，我们来看一个具体的例子。有一个计算平方的函数 $f(x) = x^2$ 。这时设 $y = f(x)$ ，那么变量 y 和 x 之间的关系就由函数 f 决定。换言之，“ y 是 x 的平方”的关系是由函数 f 决定的。

从这个例子可以看出，函数具有定义变量之间对应关系的作用。图 2-1 是变量与函数的关系示意图。

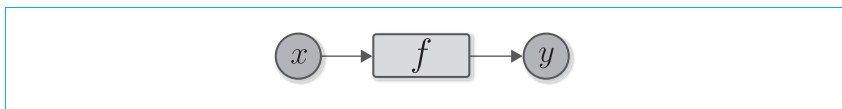


图 2-1 变量和函数的关系示意图

图 2-1 直观地展示了变量 x 和 y 与函数 f 之间的关系。这种用节点和箭头来表示计算的图叫作计算图。本书用圆框表示变量，用方框表示函数。



提到图，大家可能会想到柱状图、饼状图等。不过在计算机科学领域，图是由节点和边组成的数据结构。

2.2 Function 类的实现

下面我们从编程的角度思考图 2-1 表示的函数。具体来说，就是假设变量 x 和 y 是之前实现的 `Variable` 实例，然后以 `Function` 类的形式实现可以处理它们的函数 f 。这里有两点需要注意。

- 在 `Function` 类中实现的方法，其输入应为 `Variable` 实例，输出应为 `Variable` 实例
- `Variable` 实例的实际数据存在于实例变量 `data` 中

在满足这两点的基础上，将 `Function` 类按以下方式实现。

```
class Function:
    def __call__(self, input):
        x = input.data # 取出数据
        y = x ** 2 # 实际的计算
        output = Variable(y) # 作为 Variable 返回
        return output
```

上面的代码实现了 `__call__` 方法。`__call__` 方法接收 `input` 参数，这里假定传来的 `input` 是 `Variable` 实例。因此，实际数据存放在 `input.data` 中。取出数据后，方法会执行相应的计算（在这个例子中是求平方），然后将结果放到 `Variable` “箱子” 里并返回。



`__call__` 方法是一个特殊的Python方法。定义了这个方法后，当 `f = Function()` 时，就可以通过编写 `f(...)` 来调用 `__call__` 方法了。

2.3 使用Function类

现在我们来实际使用一下 `Function` 类。这里将 `Variable` 实例的 `x` 输入 `Function` 实例的 `f` 中。

```
x = Variable(np.array(10))
f = Function()
y = f(x)

print(type(y)) # 使用type(), 获取对象的类型
print(y.data)
```

运行结果

```
<class '__main__.Variable'>
100
```

上面的代码把 `Variable` 和 `Function` 结合起来使用了。从运行结果可以看出，`y` 的类型是 `Variable`，其数据存储在 `y.data` 中。

这里实现的 `Function` 类是一个“对输入值求平方”的具体函数。因此，将其命名为 `Square` 这样的具体名称比较合适。此外，今后我们还将增加各种函数（如 `Sin` 函数和 `Exp` 函数等）。考虑到这一点，最好把 `Function` 类作为基类来实现，并在这个类的内部实现所有 `DeZero` 函数都有的功能。这里，我们重新设计 `DeZero` 的函数，以满足以下两点要求。

- `Function` 类是基类，实现所有函数通用的功能
- 具体函数是在继承了 `Function` 类的类中实现的

为了满足这两点，我们将 `Function` 类按下面的方式实现。

steps/step02.py

```
class Function:
    def __call__(self, input):
        x = input.data
        y = self.forward(x) # 具体的计算在 forward 方法中进行
        output = Variable(y)
        return output

    def forward(self, x):
        raise NotImplementedError()
```

本节实现了两个方法：`__call__` 和 `forward`。`__call__` 方法执行两项任务：从 `Variable` 中取出数据和将计算结果保存到 `Variable` 中。其中具体的计算是通过调用 `forward` 方法完成的。`forward` 方法的实现会在继承类中完成。



`Function` 类的 `forward` 方法会抛出一个异常，目的是告诉使用了 `Function` 类的 `forward` 方法的人，这个方法应该通过继承来实现。

下面实现一个继承自 `Function` 类并对输入值进行平方的类。这个类的名字是 `Square`，代码如下所示。

steps/step02.py

```
class Square(Function):
    def forward(self, x):
        return x ** 2
```

`Square` 类继承自 `Function` 类，所以也继承了 `__call__` 方法。因此，我们只需在 `forward` 方法中编写具体的计算代码，就能完成 `Square` 类的实现。接下来使用 `Square` 类对 `Variable` 进行如下处理。

steps/step02.py

```
x = Variable(np.array(10))
f = Square()
y = f(x)
print(type(y))
print(y.data)
```

运行结果

```
<class '__main__.Variable'>
100
```

可以看出，得到的结果和之前的一样。这样我们就完成了[步骤2](#)，实现了 `Variable` 和 `Function` 类的基本功能。



我们暂时只考虑 `Function` 的输入和输出仅有一个变量的情况。从[步骤11](#)开始，我们将扩展 `DeZero`，以支持多个变量。

步骤3

函数的连续调用

目前已经创建了DeZero的变量和函数。我们还在上一个步骤实现了一个名为Square的用于计算平方的函数类。在本步骤中，我们会实现一个新的函数，并将多个函数连结起来进行计算。

3.1 Exp函数的实现

首先在DeZero中实现一个新函数。这里我们要实现的是 $y = e^x$ 的计算（其中， e 是自然对数，具体值为 $e = 2.718\dots$ ）。代码如下所示。

steps/step03.py

```
class Exp(Function):  
    def forward(self, x):  
        return np.exp(x)
```

与Square类的实现过程一样，继承Function类，并在forward方法中实现要计算的内容。与Square类唯一不同的是，forward方法的内容由 $x ** 2$ 变成了 $\text{np.exp}(x)$ 。

3.2 函数的连续调用

Function类的`__call__`方法的输入和输出都是Variable实例。因此，DeZero函数自然可以连续使用。比如 $y = (e^x)^2$ 的计算，我们可以像下面这样编写它的计算代码。

steps/step03.py

```
A = Square()
B = Exp()
C = Square()

x = Variable(np.array(0.5))
a = A(x)
b = B(a)
y = C(b)
print(y.data)
```

运行结果

1.648721270700128

上面的代码连续使用了A、B、C这3个函数。这里的关键点是中途出现的4个变量x、a、b、y都是Variable实例。由于Function类的`__call__`方法的输入和输出都是Variable实例，所以可以像上面的代码一样连续使用多个函数。如图3-1所示，这里进行的计算可以用函数和变量交替排列的计算图来表示。

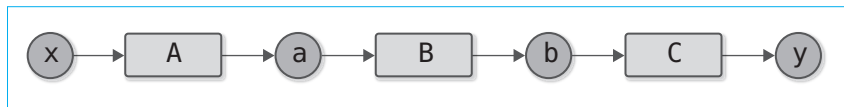


图3-1 使用多个函数的计算图(圆框代表变量，方框代表函数)



我们可以把图3-1那种依次应用多个函数创建的变换看作一个大的函数。这种由多个函数组成的函数叫作复合函数。值得注意的是，即使组成复合函数的每个函数都是简单的计算，我们也可以依次连续使用它们来执行复杂的计算。

为什么要用计算图来表示一系列的计算呢？答案是这样能高效地求出每个变量的导数（准确来说，是做好了这样的准备）。这个算法就是**反向传播**（**back propagation**）。从下一个步骤开始，我们将扩展 DeZero，以支持反向传播。

步骤4

数值微分

现在，我们已经实现了 `Variable` 类和 `Function` 类。实现这些类的目的是自动微分。在本步骤中，我们会先复习导数，并尝试用一种叫作数值微分 (numerical differentiation) 的方法来求导。从下一个步骤开始，我们将实现一种替代数值微分的更高效的算法——**反向传播**。



导数不仅仅在机器学习领域，在其他领域也很重要。从流体力学、金融工程到气象模拟、工程设计优化，很多领域需要进行导数计算，而且这些领域实际上都在使用自动微分的功能。

4.1 什么是导数

什么是导数？简单地说，导数是变化率的一种表示方式。比如某个物体的位置相对于时间的变化率就是位置的导数，即速度。速度相对于时间的变化率就是速度的导数，即加速度。像这样，导数表示的是变化率，它被定义为在极短时间内的变化量。函数 $f(x)$ 在 x 处的导数可用下面的式子表示。

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \quad (4.1)$$

式子 4.1 中的 $\lim_{h \rightarrow 0}$ 表示极限，意思是 h 应尽可能地接近 0。式子 4.1 中的

$\frac{f(x+h)-f(x)}{h}$ 为图4-1中通过两点的直线的斜率。

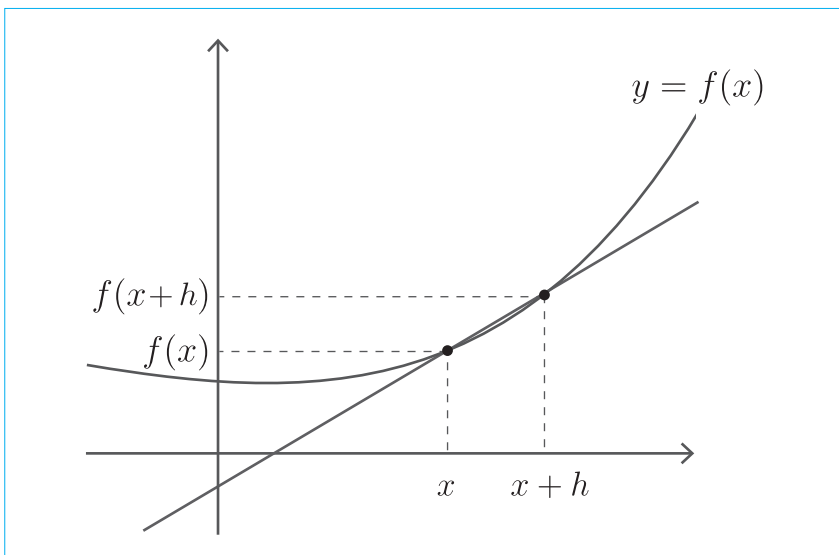


图4-1 曲线 $y = f(x)$ 和通过其两点的直线

如图4-1所示，函数 $f(x)$ 在 x 和 $x+h$ 两点之间的变化率为 $\frac{f(x+h)-f(x)}{h}$ 。让 h 的值尽可能地接近0，就可以求出 x 处的变化率。这就是 $y = f(x)$ 的导数。另外，在 $y = f(x)$ 的可导区间内，对于该区间内的任何 x ，式子4.1都成立。因此，式子4.1中的 $f'(x)$ 也是一个函数，我们称之为 $f(x)$ 的导函数。

4.2 数值微分的实现

下面根据导数的定义式，即式子4.1来实现求导。需要注意的是，计算机不能处理极限值。因此，这里的 h 表示一个近似值。例如我们可以用 $h = 0.0001 (= 1e-4)$ 这种非常小的值来计算式子4.1。利用微小的差值获得函数变化量的方法叫作数值微分。

数值微分使用非常小的值 h 求出的是真的导数的近似值。因此，这个值

包含误差。中心差分近似是减小近似值误差的一种方法。中心差分近似计算的不是 $f(x)$ 和 $f(x+h)$ 的差，而是 $f(x-h)$ 和 $f(x+h)$ 的差。图 4-2 中的粗线表示的就是中心差分近似。

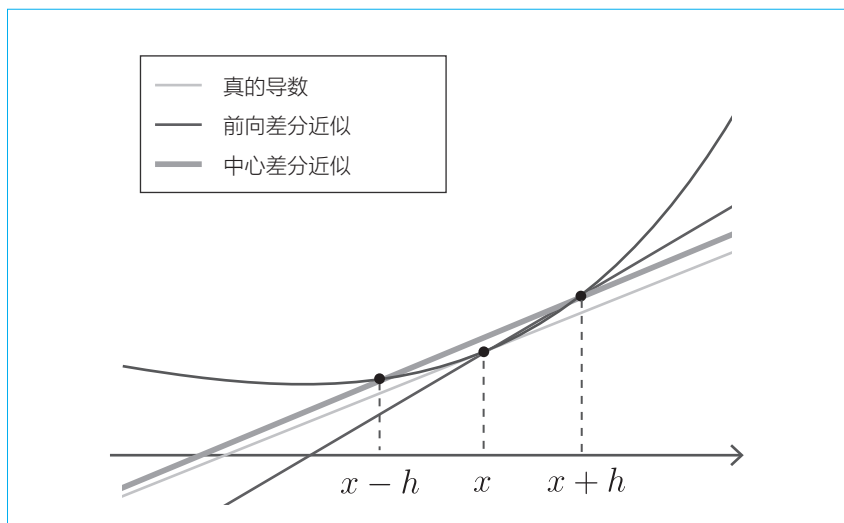


图 4-2 比较真的导数、前向差分近似和中心差分近似

如图 4-2 所示，计算过 x 和 $x+h$ 这两点的直线的斜率的方法称为前向差分近似，计算 $x-h$ 和 $x+h$ 这两点间斜率的方法称为中心差分近似，中心差分近似实际产生的误差更小。这一点的证明超出了本书的范围，不过我们从图 4-2 中直线的斜率可以直观地得出这个结论。中心差分近似的直线斜率为 $\frac{f(x+h)-f(x-h)}{2h}$ (注意分母是 $2h$)。



中心差分近似比前向差分近似更接近真的导数这个结论可以通过泰勒展开来证明。相关证明过程请阅读参考文献 [1] 等资料。

下面我们来实现使用中心差分近似求数值微分的函数，该函数的名称为

`numerical_diff(f, x, eps=1e-4)`。这里的参数 `f` 是被求导的函数，是 `Function` 类的实例。参数 `x` 为求导的变量，是 `Variable` 类的实例。参数 `eps` 代表一个微小的值，默认值是 `1e-4` ^① (`eps` 是 `epsilon` 的缩写)。数值微分可通过以下代码实现。

steps/step04.py

```
def numerical_diff(f, x, eps=1e-4):
    x0 = Variable(x.data - eps)
    x1 = Variable(x.data + eps)
    y0 = f(x0)
    y1 = f(x1)
    return (y1.data - y0.data) / (2 * eps)
```

只要注意 `Variable` 实例变量 `data` 中包含实际数据即可。下面我们使用这个函数求步骤3中实现的 `Square` 类的导数。

steps/step04.py

```
f = Square()
x = Variable(np.array(2.0))
dy = numerical_diff(f, x)
print(dy)
```

运行结果

4.000000000004

由上面的运行结果可知， $y = x^2$ 在 $x = 2.0$ 时计算得到的导数值为 `4.000000000004`。不包含误差的导数值是 `4.0`，可以说这个结果大体正确。



导数也可以通过解析解的方式求解。解析解的方式求解是指只通过式子的变形推导出答案。在上面的例子中，根据导数的公式可知， $y = x^2$ 的导数为 $\frac{dy}{dx} = 2x$ ($\frac{dy}{dx}$ 是 y 对 x 求导的符号)。因此， $y = x^2$ 在 $x = 2.0$ 处的导数为 `4.0`。这个 `4.0` 是不包含误差的值。前面的数值微分结果虽然不是正好为 `4.0`，但我们可以看出误差是相当小的。

^① `1e-4` 表示 `0.0001`。

4.3 复合函数的导数

我们在前面接触的是 $y = x^2$ 这种简单的函数。接下来尝试对复合函数求导。下面求 $y = (e^{x^2})^2$ 的导数 $\frac{dy}{dx}$ 。代码如下所示。

steps/step04.py

```
def f(x):  
    A = Square()  
    B = Exp()  
    C = Square()  
    return C(B(A(x)))  
  
x = Variable(np.array(0.5))  
dy = numerical_diff(f, x)  
print(dy)
```

运行结果

3.2974426293330694

上面的代码将一系列的计算组合成了一个名为 `f` 的函数。函数在 Python 中也是对象，所以可以作为参数传给其他函数。在上面的例子中，函数 `f` 作为参数传给了 `numerical_diff` 函数。

运行上面的代码，得到的导数是 3.297...。这意味着，如果 x 从 0.5 变成一个微小的值， y 值的变化幅度将是这个微小值的 3.297... 倍。

现在我们已经成功实现了“自动”求导。只要用代码来定义要完成的计算（前面的例子定义了函数 `f`），程序就会自动求出导数。使用这种方法，无论多么复杂的函数组合，程序都能自动求出导数。今后函数的种类越来越多的话，不管是什么计算，只要是可微函数，程序就能求出它的导数。不过遗憾的是，数值微分的方法存在一些问题。

4.4 数值微分存在的问题

数值微分的结果包含误差。在多数情况下，这个误差非常小，但在一些情况下，计算产生的误差可能会很大。



数值微分的结果中容易包含误差的主要原因是“精度丢失”。中心差分近似等求差值的方法计算的是相同量级数值之间的差，但由于精度丢失，计算结果中会出现有效位数减少的情况。以有效位数为4的情况为例，在计算两个相近的值之间的差时，比如 $1.234 - 1.233$ ，其结果为0.001，有效位数只有1位。本来可能是 $1.234\dots - 1.233\dots = 0.001434\dots$ 之类的结果，但由于精度丢失，结果变成0.001。同样的情况也会发生在数值微分的差值计算中，精度丢失使结果更容易包含误差。

数值微分更严重的问题是计算成本高。具体来说，在求多个变量的导数时，程序需要计算每个变量的导数。有些神经网络包含几百万个以上的变量（参数），通过数值微分对这么多的变量求导是不现实的。这时，反向传播就派上了用场。从下一个步骤开始，笔者将介绍反向传播。

另外，数值微分可以轻松实现，并能计算出大体正确的数值。而反向传播是一种复杂的算法，实现时容易出现bug。我们可以使用数值微分的结果检查反向传播的实现是否正确。这种做法叫作**梯度检验 (gradient checking)**，它是一种将数值微分的结果与反向传播的结果进行比较的方法。[步骤10](#)实现了梯度检验。

步骤5

反向传播的理论知识

我们通过数值微分成功求出了导数。但是，数值微分在计算成本和精度方面存在问题。反向传播可以解决这两个问题。也就是说，反向传播不仅能高效地求导，还能帮助我们得到误差更小的值。本步骤只介绍反向传播的理论知识，不对其进行实现。从下一个步骤开始，我们再去实现反向传播。

5.1 链式法则

理解反向传播的关键是**链式法则 (连锁律)**。链(chain)可以理解为链条、锁链等，在这里表示多个函数连接在一起使用。链式法则意为连接起来的多个函数(复合函数)的导数可以分解为各组成函数的导数的乘积。

下面看一个链式法则的具体例子。假设有一个函数 $y = F(x)$ ，这个函数 F 由3个函数组成： $a = A(x)$ 、 $b = B(a)$ 和 $y = C(b)$ 。该函数的计算图如图5-1所示。

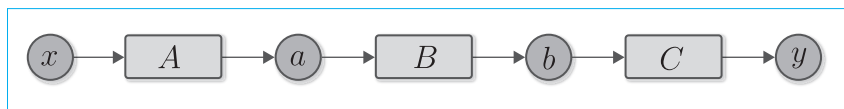


图5-1 复合函数的例子

这时， y 对 x 的导数可以用式子5.1表示。

$$\frac{dy}{dx} = \frac{dy}{db} \frac{db}{da} \frac{da}{dx} \quad (5.1)$$

如式子5.1所示， y 对 x 的导数可以表示为各函数的导数的乘积。换言之，复合函数的导数可以分解为各组成函数导数的乘积。这就是链式法则。式子5.1所表示的链式法则也可以像下面这样写成包含 $\frac{dy}{dy}$ 的形式。

$$\frac{dy}{dx} = \frac{dy}{dy} \frac{dy}{db} \frac{db}{da} \frac{da}{dx} \quad (5.2)$$

$\frac{dy}{dy}$ 是对自身求导，导数值永远为1。计算时通常省略 $\frac{dy}{dy}$ 这种针对自身的导数，但考虑到反向传播的实现，这里特意加上了这一项。



$\frac{dy}{dy}$ 是 y 对 y 的导数。即使 y 变化的幅度很小， y 自身也会变化同样大小的值。因此，不管对于什么函数，变化率总是1。

5.2 反向传播的推导

下面仔细观察式子5.2。式子5.2表示复合函数的导数可以分解为各函数导数的乘积。但是，它并没有规定各导数相乘的顺序。当然，这一点我们可以自由决定。这里，我们按照式子5.3的方式以输出到输入的顺序进行计算^①。

$$\frac{dy}{dx} = \left(\left(\frac{dy}{dy} \frac{dy}{db} \right) \frac{db}{da} \right) \frac{da}{dx} \quad (5.3)$$

式子5.3按照从输出到输入的顺序进行导数的计算，计算方向与平时相反。这时，式子5.3的计算流程如图5-2所示。

① 我们也可以考虑改变括号的位置使计算按照输入到输出的方向依次进行。这种计算方法叫作“前向模式的自动微分”。关于前向模式的自动微分，请阅读步骤10的专栏部分。

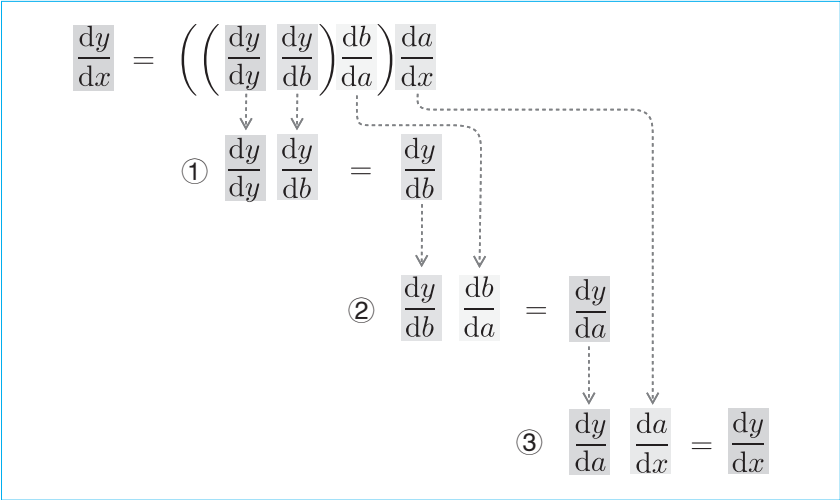


图 5-2 从输出端的导数开始依次进行计算的流程 (参见彩图)

在图 5-2 中, 导数是按照从输出 y 到输入 x 的方向依次相乘计算得出的。通过这种方法, 最终得到 $\frac{dy}{dx}$ 。图 5-3 是相应的计算图。

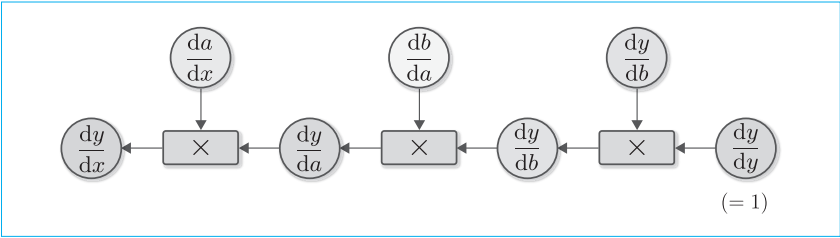


图 5-3 求 $\frac{dy}{dx}$ 的计算图

下面仔细观察图 5-3。我们先从 $\frac{dy}{dy} (= 1)$ 开始, 计算它与 $\frac{dy}{db}$ 的乘积。这里的 $\frac{dy}{db}$ 是函数 $y=C(b)$ 的导数。因此, 如果用 C' 表示函数 C 的导函数, 我们就可以把式子写成 $\frac{dy}{db} = C'(b)$ 。同样, 有 $\frac{db}{da} = B'(a)$, $\frac{da}{dx} = A'(x)$ 。基于以上内容, 图 5-3 可以简化成图 5-4。

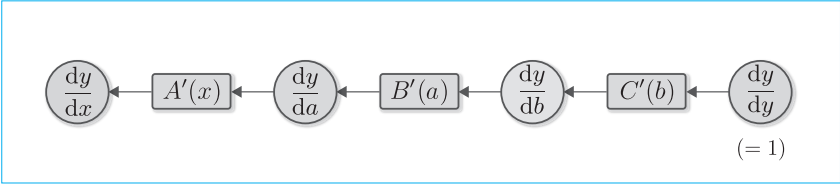


图 5-4 简化后的反向传播计算图 ($A'(x)$ 的乘法在图中简化表示为节点 $A'(x)$)

图 5-4 中把导函数和乘号合并表示为一个函数节点。这样导数计算的流程就明确了。从图 5-4 中可以看出，“ y 对各变量的导数”从右向左传播。这就是反向传播。这里重要的一点是传播的数据都是 y 的导数。具体来说，就是 $\frac{dy}{dy}$ 、 $\frac{dy}{db}$ 、 $\frac{dy}{da}$ 和 $\frac{dy}{dx}$ 这种“ y 对 $\times \times$ 变量的导数”在传播。



像式子 5.3 那样将计算顺序规定为从输出到输入，是为了传播 y 的导数。换言之，就是把 y 当作“重要人物”。如果按照从输入到输出的顺序计算，输入 x 就是“重要人物”。在这种情况下，传播的导数将是 $\frac{dx}{dx} \rightarrow \frac{da}{dx} \rightarrow \frac{db}{dx} \rightarrow \frac{dy}{dx}$ 这种形式，也就是对 x 的导数进行传播。

许多机器学习问题采用了以大量参数作为输入，以损失函数作为最终输出的形式。损失函数的输出（在许多情况下）是一个标量值，它是“重要人物”。这意味着我们需要找到损失函数对每个参数的导数。在这种情况下，如果沿着从输出到输入的方向传播导数，只要传播一次，就能求出对所有参数的导数。因为该方法的计算效率较高，所以我们采用反向传播导数的方式。

5.3 用计算图表示

下面我们把正向传播的计算图(图 5-1)和反向传播的计算图(图 5-4)以上下排列的方式画出来。

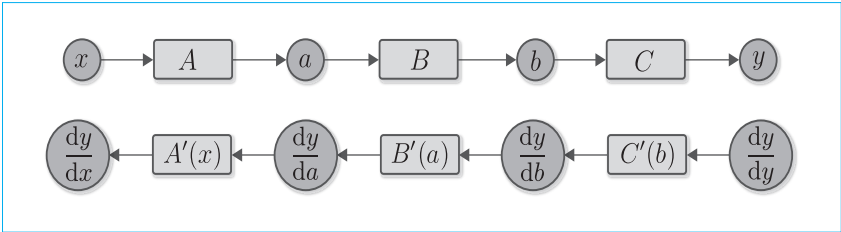


图 5-5 正向传播和反向传播

从图 5-5 可以看出，正向传播和反向传播之间存在明确的对应关系。例如，正向传播时的变量 a 对应于反向传播时的导数 $\frac{dy}{da}$ 。同样， b 对应于 $\frac{dy}{db}$ ， x 对应于 $\frac{dy}{dx}$ 。我们也可以看出函数之间存在对应关系。例如，函数 B 的反向传播对应于 $B'(a)$ ， A 对应于 $A'(x)$ 。这样一来，我们可以认为变量有普通值和导数值，函数有普通计算（正向传播）和求导计算（反向传播）。于是，反向传播就设计好了。

最后来关注一下图 5-5 中 $C'(b)$ 的函数节点。它是 $y = C(b)$ 的导数，但要注意的是，计算 $C'(b)$ 需要用到 b 的值。同理，要计算 $B'(a)$ 就得输入 a 的值。这意味着进行反向传播时需要用到正向传播中使用的数据。因此，在实现反向传播时，需要先进行正向传播，并且存储各函数输入的变量值，也就是前面例子中的 x 、 a 和 b ，之后就能对每个函数进行反向传播的计算了。

以上就是反向传播理论知识的相关内容，这是本书的难点之一。大家现在可能还没完全弄明白，但是实际运行代码后，就会理解得越来越透彻。在下一个步骤，我们将实现反向传播，并通过实际运行代码来验证它。

步骤6

手动进行反向传播

上一个步骤介绍了反向传播的机制。本步骤将扩展现有的Variable类和Function类，实现通过反向传播来求导的功能。首先是Variable类。

6.1 Variable类的功能扩展

下面实现支持反向传播的Variable类。为此，我们要扩展Variable类，除普通值(data)之外，增加与之对应的导数值(grad)。阴影部分是新增加的代码。

steps/step06.py

```
class Variable:
    def __init__(self, data):
        self.data = data
        self.grad = None
```

上面的代码在类中增加了一个新的实例变量grad。实例变量data和grad都被设置为NumPy的多维数组(ndarray)。另外，grad被初始化为None，我们要在通过反向传播实际计算导数时将其设置为求出的值。



梯度(gradient)是对包含多个变量的向量和矩阵等求导的导数。因此Variable类中增加了一个名为grad的变量，grad是gradient的缩写。

6.2 Function类的功能扩展

接下来是Function类。在前面的步骤中，Function类实现了进行普通计算的正向传播（forward方法）的功能。在此基础上，我们新增以下两个功能。

- 计算导数的反向传播（backward方法）功能
- 调用forward方法时，保有被输入的Variable实例的功能

下面的代码实现了这两个功能。

steps/step06.py

```
class Function:
    def __call__(self, input):
        x = input.data
        y = self.forward(x)
        output = Variable(y)
        self.input = input # 保存输入的变量
        return output

    def forward(self, x):
        raise NotImplementedError()

    def backward(self, gy):
        raise NotImplementedError()
```

在上面的代码中，__call__方法将输入的input设置为实例变量。这样一来，当调用backward方法时，向函数输入的Variable实例就可以作为self.input使用。

6.3 Square类和Exp类的功能扩展

接下来实现具体函数的反向传播（backward）。首先从计算平方的Square类开始。由于 $y = x^2$ 的导数是 $\frac{dy}{dx} = 2x$ ，所以这个类可以按照如下方式实现。

steps/step06.py

```
class Square(Function):
    def forward(self, x):
        y = x ** 2
        return y

    def backward(self, gy):
        x = self.input.data
        gx = 2 * x * gy
        return gx
```

上面的代码增加了用于反向传播的 `backward` 方法。这个方法的参数 `gy` 是一个 `ndarray` 实例，它是从输出传播而来的导数。`backward` 返回的结果是通过这个参数传播来的导数和“ $y = x^2$ 的导数”的乘积。这个返回结果会进一步向输入方向传播。

接下来是计算 $y = e^x$ 的 `Exp` 类。由于 $\frac{dy}{dx} = e^x$ ，所以这个类可以按下面的方式实现。

steps/step06.py

```
class Exp(Function):
    def forward(self, x):
        y = np.exp(x)
        return y

    def backward(self, gy):
        x = self.input.data
        gx = np.exp(x) * gy
        return gx
```

6.4 反向传播的实现

这样就做好准备工作了。下面我们尝试通过反向传播对图 6-1 的计算求导。

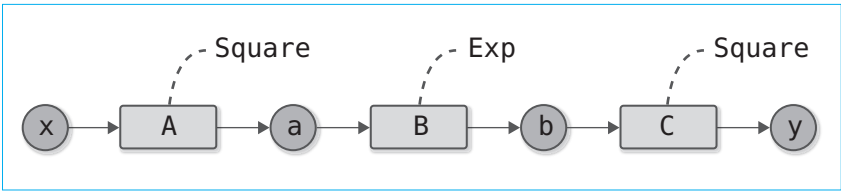


图6-1 进行反向传播的复合函数

首先编写图6-1所示的正向传播的代码。

steps/step06.py

```
A = Square()
B = Exp()
C = Square()

x = Variable(np.array(0.5))
a = A(x)
b = B(a)
y = C(b)
```

接着通过反向传播计算y的导数。为此，我们需要按照与正向传播相反的顺序调用各函数的backward方法。图6-2是这个反向传播计算的计算图。

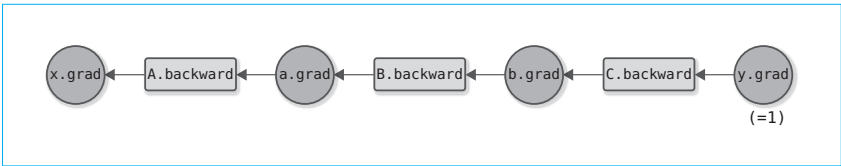


图6-2 反向传播的计算图

从图6-2可以看出各个函数的backward方法的调用顺序，也能看出应该将backward方法的结果赋给哪个变量的grad。下面是反向传播的实现。

steps/step06.py

```
y.grad = np.array(1.0)
b.grad = C.backward(y.grad)
a.grad = B.backward(b.grad)
x.grad = A.backward(a.grad)
print(x.grad)
```

运行结果

3.297442541400256

反向传播从 $\frac{dy}{dy} = 1$ 开始。因此，我们将输出 y 的导数设为 `np.array(1.0)`。之后，只要按照 $C \rightarrow B \rightarrow A$ 的顺序调用 `backward` 方法即可。这样就可以对各变量求出导数。

运行上面的代码后，得到的 `x.grad` 的结果是 `3.297442541400256`。这就是 y 对 x 的导数。顺带一提，[步骤4](#) 的数值微分的结果是 `3.2974426293330694`，这两个结果几乎一样。这说明反向传播的实现是正确的，更准确地说，这个实现大概率是正确的。

这样就完成了反向传播的实现。虽然我们得到了正确的运行结果，但是反向传播的顺序 $C \rightarrow B \rightarrow A$ 是通过编码手动指定的。在下一个步骤，我们会把这项工作自动化。

步骤7

反向传播的自动化

在上一个步骤中，我们实现的反向传播成功运行。但是，我们不得不手动编写进行反向传播计算的代码。这就意味着每次进行新的计算时，都得编写这部分代码。比如在图7-1所示的情况下，我们必须为每个计算图编写反向传播的代码。这样不但容易出错，还浪费时间，所以我们让Python来做这些无聊的事情吧。

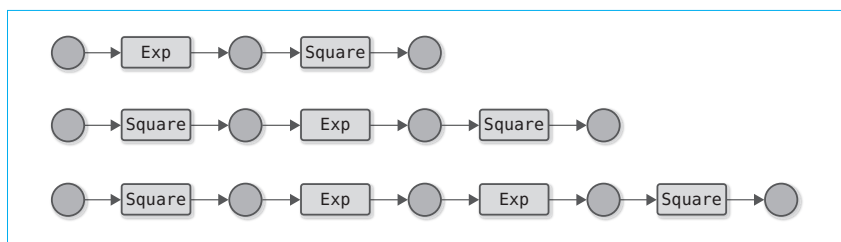


图7-1 各种计算图的例子(变量名省略，函数用类名表示)

接下来要做的就是让反向传播自动化。准确来说，就是要建立这样一个机制：无论普通的计算流程(正向传播)中是什么样的计算，反向传播都能自动进行。我们马上要接触到 Define-by-Run 的核心了。



Define-by-Run是在深度学习中进行计算时，在计算之间建立“连接”的机制。这种机制也称为动态计算图。关于 Define-by-Run 及其优点的详细信息，请阅读[步骤24](#)的专栏。

[图7-1](#)所示的计算图都是流水线式的计算。因此，只要以列表的形式记录函数的顺序，就可以通过反向回溯自动进行反向传播。不过，对于有分支的计算图或多次使用同一个变量的复杂计算图，只借助简单的列表就不能奏效了。我们接下来的目标是建立一个不管计算图多么复杂，都能自动进行反向传播的机制。



其实只要在列表的数据结构上想想办法，将所做的计算添加到列表中，或许可以对任意的计算图准确地进行反向传播。这种数据结构叫作 Wengert 列表（也叫 tape）。本书不对 Wengert 列表进行说明，感兴趣的读者请参阅参考文献[2]和参考文献[3]等。另外，关于借助 Wengert 列表实现 Define-by-Run 的优点，请阅读参考文献[4]等。

7.1 为反向传播的自动化创造条件

在实现反向传播的自动化之前，我们先思考一下变量和函数之间的关系。首先从函数的角度来考虑，即思考“从函数的角度如何看待变量”。从函数的角度来看，变量是以输入和输出的形式存在的。如[图7-2](#)左图所示，函数的变量包括“输入变量”(input)和“输出变量”(output)(图中的虚线表示引用)。

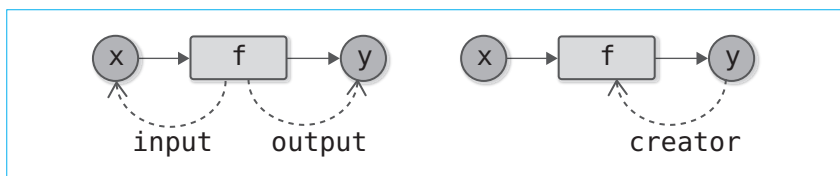


图7-2 从函数的角度来看其与变量的关系(左图)和从变量的角度来看其与函数的关系(右图)

那么从变量的角度来看，函数是什么样的呢？这里要强调的是变量是由函数“创造”的。也就是说，函数是变量的“父母”，是creator(创造者)。如果变量没有作为创造者的函数，我们就可以认为它是由非函数创造的，比如用户给出的变量。

下面在代码中实现图7-2所示的函数和变量之间的“连接”。我们让这个“连接”在执行普通计算(正向传播)的那一刻创建。为此，先在Variable类中添加以下代码。

steps/step07.py

```
class Variable:
    def __init__(self, data):
        self.data = data
        self.grad = None
        self.creator = None

    def set_creator(self, func):
        self.creator = func
```

上面的代码添加了一个名为creator的实例变量，之后添加了用于设置creator的set_creator方法。下面我们在Function类中添加以下代码。

steps/step07.py

```
class Function:
    def __call__(self, input):
        x = input.data
        y = self.forward(x)
        output = Variable(y)
        output.set_creator(self) # 让输出变量保存创造者信息
        self.input = input
        self.output = output # 也保存输出变量
        return output
```

上面的代码通过正向传播的计算，创建了一个名为output的Variable实例。对于创建的output变量，代码让它保存了“我(函数本身)是创造者”的信息。这是动态建立“连接”这一机制的核心。为了兼顾下一个步骤，这里将输出

设置为实例变量 `output`。



DeZero的动态计算图的原理是在执行实际的计算时，在变量这个“箱子”里记录它的“连接”。Chainer和PyTorch也采用了类似的机制。

这样一来，`Variable`和`Function`之间就有了“连接”，我们就可以反向遍历计算图了。具体的实现代码如下所示。

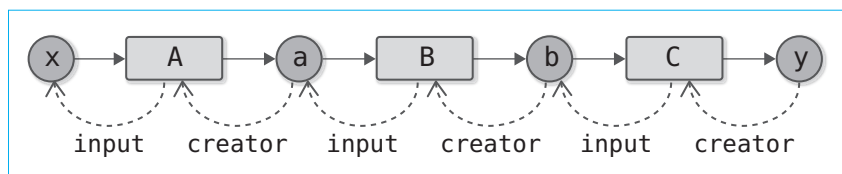
```
A = Square()
B = Exp()
C = Square()

x = Variable(np.array(0.5))
a = A(x)
b = B(a)
y = C(b)

# 反向遍历计算图的节点
assert y.creator == C
assert y.creator.input == b
assert y.creator.input.creator == B
assert y.creator.input.creator.input == a
assert y.creator.input.creator.input.creator == A
assert y.creator.input.creator.input.creator.input == x
```

首先介绍一下 `assert` (断言) 语句，它的用法是 `assert...`。如果这里的...不为 `True`，就会抛出异常。因此可以使用 `assert` 语句来检查条件是否得到满足。上面的代码在运行时没有发生任何问题(没有抛出异常)，这意味着 `assert` 语句的所有条件都得到了满足。

上面的代码通过 `Variable` 实例变量 `creator` 找到前一个 `Function`，然后通过 `Function` 的实例变量 `input` 找到前一个 `Variable`。它们的连接方式如图 7-3 所示。

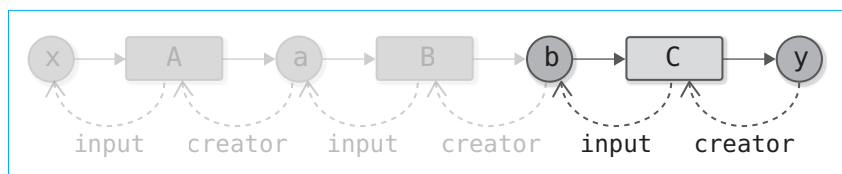
图7-3 以 y 为起点反向遍历计算图

如图7-3所示，计算图是由函数和变量之间的“连接”构建而成的。更重要的是，这个“连接”是在计算实际发生的时候（数据在正向传播中流转的时候）形成的。变量和函数连接的这个特征就是 Define-by-Run。换言之，“连接”是通过数据的流转建立起来的。

图7-3这种带有“连接”的数据结构叫作连接节点。节点是构成图的一个元素，连接则代表对另一个节点的引用。也就是说，我们用了一个叫作“连接节点”的数据结构来表示计算图。

7.2 尝试反向传播

下面利用变量和函数之间的连接，尝试实现反向传播。首先实现从 y 到 b 的反向传播。代码如下所示（为了便于理解，这里附上了计算图）。



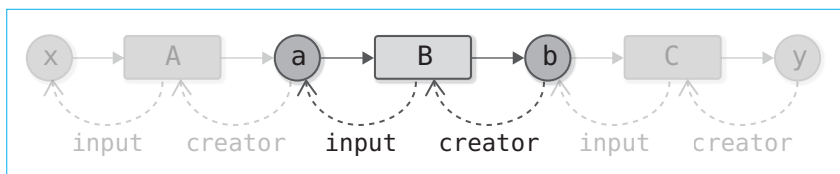
```
y.grad = np.array(1.0)
```

```
C = y.creator # 1. 获取函数
```

```
b = C.input # 2. 获取函数的输入
```

```
b.grad = C.backward(y.grad) # 3. 调用函数的backward方法
```

上面的代码从 `y` 的实例变量 `creator` 获取函数，从函数的 `input` 获取输入变量，然后调用函数的 `backward` 方法。下面是从变量 `b` 到变量 `a` 反向传播的计算图和代码。



```

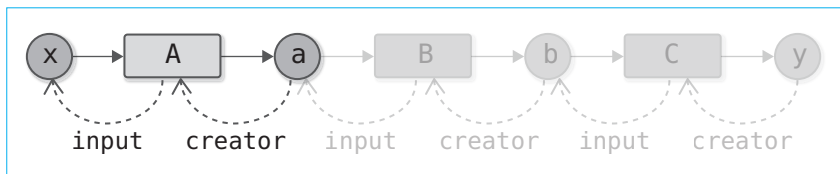
B = b.creator # 1. 获取函数
a = B.input   # 2. 获取函数的输入
a.grad = B.backward(b.grad) # 3. 调用函数的backward方法

```

上述代码执行的反向传播的逻辑与之前的相同。具体来说，该流程如下所示。

1. 获取函数
2. 获取函数的输入
3. 调用函数的 `backward` 方法

最后是从变量 `a` 到变量 `x` 的反向传播。



```

A = a.creator # 1. 获取函数
x = A.input   # 2. 获取函数的输入
x.grad = A.backward(a.grad) # 3. 调用函数的backward方法
print(x.grad)

```

运行结果

3.297442541400256

这样就完成了所有的反向传播。

7.3 增加 backward 方法

从前面这些反向传播的代码可以看出，它们有着相同的处理流程。准确来说，是从一个变量到前一个变量的反向传播逻辑相同。为了自动完成这些重复的处理，我们在 `Variable` 类中添加一个新的方法——`backward`。

steps/step07.py

```
class Variable:
    def __init__(self, data):
        self.data = data
        self.grad = None
        self.creator = None

    def set_creator(self, func):
        self.creator = func

    def backward(self):
        f = self.creator # 1. 获取函数
        if f is not None:
            x = f.input # 2. 获取函数的输入
            x.grad = f.backward(self.grad) # 3. 调用函数的 backward 方法
            x.backward() # 调用自己前面那个变量的 backward 方法(递归)
```

`backward` 方法和此前反复出现的流程基本相同。具体来说，它从 `Variable` 的 `creator` 获取函数，并取出该函数的输入变量，然后调用函数的 `backward` 方法。最后，它会针对自己前面的变量，调用它的 `backward` 方法。这样每个变量的 `backward` 方法就会被递归调用。



如果 `Variable` 实例的 `creator` 是 `None`，那么反向传播就此结束。这种情况意味着 `Variable` 实例是由非函数创造的，主要来自用户提供的变量。

下面使用这个新的 `Variable` 自动进行反向传播。

`steps/step07.py`

```
A = Square()
B = Exp()
C = Square()

x = Variable(np.array(0.5))
a = A(x)
b = B(a)
y = C(b)

# 反向传播
y.grad = np.array(1.0)
y.backward()
print(x.grad)
```

运行结果

3.297442541400256

只要像上面那样调用变量 `y` 的 `backward` 方法，反向传播就会自动进行。运行结果和之前的一样。这样我们就打好了对 DeZero 来说最重要的自动微分的基础。

步骤8

从递归到循环

在上一个步骤中，我们向 `Variable` 类添加了 `backward` 方法。考虑到处理效率的改善和今后的功能扩展，本步骤将改进 `backward` 方法的实现方式。

8.1 现在的 `Variable` 类

再来看一下前面实现的 `Variable` 类的 `backward` 方法。

steps/step07.py

```
class Variable:
    ... # 省略代码

    def backward(self):
        f = self.creator
        if f is not None:
            x = f.input
            x.grad = f.backward(self.grad)
            x.backward()
```

我们需要注意的一点是 `backward` 方法调用了(朝着输入方向的)前一个变量的 `backward` 方法。由此，“`backward` 方法内调用 `backward` 方法，被调用的 `backward` 方法内再次调用 `backward` 方法”的处理会不断延续下去(直到碰到一个 `self.creator` 函数为 `None` 的 `Variable` 变量)。这是一个递归结构。



本书为了节省篇幅，有时候会省略部分代码。省略的部分用“...”表示（这个省略号与出现在Python解释器的换行处的“...”含义不同）。

8.2 使用循环实现

接下来将前面的“使用递归实现”替换为“使用循环实现”。代码如下所示。

steps/step08.py

```
class Variable:
    ...

    def backward(self):
        funcs = [self.creator]
        while funcs:
            f = funcs.pop() # 获取函数
            x, y = f.input, f.output # 获取函数的输入
            x.grad = f.backward(y.grad) # backward调用backward方法

            if x.creator is not None:
                funcs.append(x.creator) # 将前一个函数添加到列表中
```

上面的实现使用了循环。关键点在于按顺序向 `funcs` 列表里添加应该处理的函数。在 `while` 循环中，通过 `funcs.pop()` 获取要处理的函数，将其作为变量 `f`，然后调用函数 `f` 的 `backward` 方法。通过 `f.input` 和 `f.output` 获取函数 `f` 的输入变量和输出变量，就可以正确地设置 `f.backward()` 的参数和返回值。



列表的 `pop` 方法会删除列表末尾的元素，并取出这个元素的值。例如，对于 `funcs = [1, 2, 3]`，如果执行 `x = funcs.pop()`，3 就会被取出，`funcs` 变为 `[1, 2]`。

8.3 代码验证

接下来使用前面的 `Variable` 类来求导。这里运行的是和上一个步骤相同的代码。

steps/step08.py

```
A = Square()
B = Exp()
C = Square()

x = Variable(np.array(0.5))
a = A(x)
b = B(a)
y = C(b)

# 反向传播
y.grad = np.array(1.0)
y.backward()
print(x.grad)
```

运行结果

3.297442541400256

和上一个步骤得到的结果一样。这样实现方式就从递归变成了循环。在步骤 15，我们将感受到循环带来的好处。届时要处理的是复杂的计算图，不过在使用循环的情况下，代码实现很容易扩展到复杂的计算图的处理，而且循环的执行效率也会变高。



每次递归调用函数时，函数都会将处理过程中的结果保留在内存中（或者说保留在栈中），然后继续处理。因此一般来说，循环的效率更高。不过，对现代计算机来说，使用少量的内存是没有问题的。有时可以通过尾递归的技巧，使递归处理能够按照循环的方式执行。

这样就完成了反向传播的基础实现。接下来，我们将扩展当前版本的 DeZero，以执行更复杂的计算。下一个步骤要做的是提高 DeZero 的易用性。

步骤9

让函数更易用

DeZero 现在可以通过反向传播进行计算了。它还拥有一项名为 Define-by-Run 的能力，可以在运行时在计算之间建立“连接”。为了使 DeZero 更加易用，本步骤将对 DeZero 的函数进行 3 项改进。

9.1 作为 Python 函数使用

此前，DeZero 中使用的函数是作为 Python 的类实现的。举例来说，在使用 Square 类进行计算的情况下，我们需要编写如下代码。

```
x = Variable(np.array(0.5))
f = Square()
y = f(x)
```

上面的代码是分两步计算平方的：创建一个 Square 类的实例；调用这个实例。但是从用户的角度来看，分两步完成有点啰唆（虽然可以写成 `y = Square()(x)`，但观感很差）。用户更希望把 DeZero 的函数当作 Python 函数使用。为此，需要添加以下代码。

```
def square(x):  
    f = Square()  
    return f(x)  
  
def exp(x):  
    f = Exp()  
    return f(x)
```

上面的代码实现了 `square` 和 `exp` 两个函数。这样我们就可以将 DeZero 的函数当作 Python 函数使用了。顺便提一下，上面的代码也可以像下面这样写成一行。

steps/step09.py

```
def square(x):  
    return Square()(x) # 编写在一行内  
  
def exp(x):  
    return Exp()(x)
```

上面的代码没有像 `f = Square()` 这样通过变量名 `f` 来引用，而是直接写成了 `Square()(x)`，这样也是可以的。接下来我们使用一下前面实现的两个函数。

```
x = Variable(np.array(0.5))  
a = square(x)  
b = exp(a)  
y = square(b)  
  
y.grad = np.array(1.0)  
y.backward()  
print(x.grad)
```

运行结果

3.297442541400256

如上所示，用 `Variable` 封装 `np.array(0.5)` 后，就可以像使用 NumPy 进行计算一样来编码了。另外，函数现在也支持连续调用，示例代码如下所示。

```
x = Variable(np.array(0.5))
y = square(exp(square(x))) # 连续调用
y.grad = np.array(1.0)
y.backward()
print(x.grad)
```

运行结果

3.297442541400256

这样我们就可以编写更自然的代码进行计算。以上是第1项改进。

9.2 简化 backward 方法

第2项要改进的地方是减少用户在反向传播方面所做的工作。具体来说，就是省略前面代码中的 `y.grad = np.array(1.0)`。每次反向传播时我们都要重新编写这行代码，为了省略它，我们需要在 `Variable` 的 `backward` 方法中添加阴影部分的两行代码。

steps/step09.py

```
class Variable:
    ...

    def backward(self):
        if self.grad is None:
            self.grad = np.ones_like(self.data)

        funcs = [self.creator]
        while funcs:
            f = funcs.pop()
            x, y = f.input, f.output
            x.grad = f.backward(y.grad)

            if x.creator is not None:
                funcs.append(x.creator)
```

如上所示，如果变量 `grad` 为 `None`，则自动生成导数。代码中通过 `np.ones_`

`like(self.data)` 创建了一个 `ndarray` 实例，该实例的形状和数据类型与 `self.data` 的相同，元素为 1。如果 `self.data` 是标量，那么 `self.grad` 也是标量。



之前的代码中输出的导数是 `np.array(1.0)`，而上面的代码使用了 `np.ones_like()`。这么编写的原因是 `Variable` 中的 `data` 和 `grad` 的数据类型是一样的。如果 `data` 的数据类型是 32 位浮点数，那么 `grad` 的数据类型也是 32 位浮点数。顺带一提，如果编写的是 `np.array(1.0)`，它的数据类型就是 64 位的浮点数。

之后如果再做某个计算，只需对最终输出的变量调用 `backward` 方法就能求得导数。下面是示例代码。

steps/step09.py

```
x = Variable(np.array(0.5))
y = square(exp(square(x)))
y.backward()
print(x.grad)
```

运行结果

3.297442541400256

9.3 只支持 `ndarray`

DeZero 的 `Variable` 只支持 `ndarray` 实例的数据。但是，有些用户很可能会不小心使用 `float` 或 `int` 等数据类型，例如 `Variable(1.0)` 和 `Variable(3)` 等。考虑到这一点，我们再做一点优化，使 `Variable` 成为只能容纳 `ndarray` 实例的“箱子”。具体来说，就是当把 `ndarray` 实例以外的数据放入 `Variable` 时，让 DeZero 立即抛出错误（`None` 是允许放入的）。这项改进有望让用户在早期发现问题。下面我们在 `Variable` 类的初始化部分添加以下代码。

steps/step09.py

```
class Variable:
    def __init__(self, data):
        if data is not None:
            if not isinstance(data, np.ndarray):
                raise TypeError('{} is not supported'.format(type(data)))

        self.data = data
        self.grad = None
        self.creator = None
```

在上面的代码中，如果作为参数的 `data` 不是 `None`，也不是 `ndarray` 实例，就会引发 `TypeError` 异常。这时，程序会输出代码中指定的字符串作为错误提示。现在，可以像下面这样使用 `Variable`。

steps/step09.py

```
x = Variable(np.array(1.0)) # OK
x = Variable(None) # OK

x = Variable(1.0) # NG: 错误发生!
```

运行结果

```
TypeError: <class 'float'> is not supported
```

在上面的代码中，如果数据为 `ndarray` 或 `None`，就可以顺利创建 `Variable`。但如果是其他的数据类型，比如上面代码中的 `float`，`DeZero` 就会抛出一个异常。这样一来，用户就能立刻知道自己使用了错误的数据类型。

这项改进也带来一个问题，这个问题是由 `NumPy` 自身的特点所导致的。在解释这个问题之前，我们先看看下面的 `NumPy` 代码。

```
x = np.array([1.0])
y = x ** 2
print(type(x), x.ndim)
print(type(y))
```

运行结果

```
<class 'numpy.ndarray'> 1
<class 'numpy.ndarray'>
```

代码中的 x 是一维的 `ndarray`。 $x ** 2$ (平方) 的结果 y 的数据类型是 `ndarray`。这是预期的结果，问题出现在下面这种情况。

```
x = np.array(1.0)
y = x ** 2
print(type(x), x.ndim)
print(type(y))
```

运行结果

```
<class 'numpy.ndarray'> 0
<class 'numpy.float64'>
```

上面代码中的 x 是零维的 `ndarray`，而 $x ** 2$ 的结果是 `np.float64`，这是 NumPy 的运行方式^①。换言之，如果用零维的 `ndarray` 实例进行计算，结果将是 `ndarray` 实例以外的数据类型，如 `numpy.float64`、`numpy.float32` 等。这意味着 DeZero 函数的输出 `Variable` 可能是 `numpy.float64` 或 `numpy.float32` 类型的数据。但是，`Variable` 中的数据只允许保存 `ndarray` 实例。为了解决这个问题，我们首先准备以下函数作为工具函数。

steps/step09.py

```
def as_array(x):
    if np.isscalar(x):
        return np.array(x)
    return x
```

上面的代码使用 `np.isscalar` 函数来检查 `numpy.float64` 等属于标量的类型（它也可以用来检查 Python 的 `int` 和 `float`）。下面是使用 `np.isscalar` 函数的示例代码。

① CuPy 的实现方法几乎与 NumPy 相同，只是在“零维 `ndarray` 的类型”这一点上不同。在使用 CuPy 时，对 `cupy.ndarray` 的计算结果始终是 `cupy.ndarray`，与维度无关。

```
>>> import numpy as np
>>> np.isscalar(np.float64(1.0))
True
>>> np.isscalar(2.0)
True
>>> np.isscalar(np.array(1.0))
False
>>> np.isscalar(np.array([1, 2, 3]))
False
```

从这些例子可以看出，通过 `np.isscalar(x)` 可以判断 `x` 是否为 `ndarray` 实例。如果不是，则使用 `as_array` 函数将其转换为 `ndarray` 实例。实现 `as_array` 这个工具函数之后，在 `Function` 类中添加以下阴影部分的代码。

steps/step09.py

```
class Function:
    def __call__(self, input):
        x = input.data
        y = self.forward(x)
        output = Variable(as_array(y))
        output.set_creator(self)
        self.input = input
        self.output = output
        return output

    ...
```

上面的代码在将正向传播的结果 `y` 封装在 `Variable` 中时使用了 `as_array(y)`，这样可以确保输出结果 `output` 是 `ndarray` 实例的数据。即使使用零维的 `ndarray` 实例进行计算，所有的数据也会是 `ndarray` 实例。

至此，本步骤的工作就完成了。下一个步骤的主题是测试 `DeZero`。

步骤 10

测试

在软件开发中，测试不可或缺。测试可以暴露错误(bug)，自动化测试可以持续保持软件的品质。我们打造的 DeZero 框架也需要测试。本步骤将介绍测试方法，尤其是深度学习框架的测试方法。



随着软件规模的扩大，软件测试的方法和规定也逐渐变多。但是不要把测试想得很难，尤其是在软件开发的早期阶段。重要的是先建立起测试的意识。本步骤要进行的不是全面的测试，而是尽可能简单的测试。

10.1 Python 的单元测试

在 Python 中进行测试的一个很方便的做法是使用标准库中的 `unittest`。下面对前面的步骤中实现的 `square` 函数进行测试。代码如下所示。

steps/step10.py

```
import unittest

class SquareTest(unittest.TestCase):
    def test_forward(self):
        x = Variable(np.array(2.0))
        y = square(x)
        expected = np.array(4.0)
        self.assertEqual(y.data, expected)
```

前面的代码首先导入了 `unittest`，然后实现了继承 `unittest.TestCase` 的 `SquareTest` 类。创建名字以 `test` 开头的方法后，将测试逻辑写在该方法中。示例中的测试用于验证 `square` 函数的输出与预期值是否一致。具体来说，它验证的是当输入为 `2.0` 时，输出是否为 `4.0`。



上面的例子使用方法 `self.assertEqual` 来验证 `square` 函数的输出与预期值是否一致。这个方法可以判断两个给定对象是否相等。除了这个方法，`unittest` 还提供了 `self.assertGreater` 和 `self.assertTrue` 等方法。关于其他方法的更多信息，请阅读参考文献[8]等资料。

现在运行上面的测试代码。假定该测试代码在文件 `step/step10.py` 中。这时，在终端运行以下命令。

```
$ python -m unittest steps/step10.py
```

像这样在命令中加上参数 `-m unittest`，就能在测试模式下运行 Python 文件了。另外，只要在 `step10.py` 文件末尾添加以下代码，就能用 `python steps/step10.py` 命令运行测试。

```
# step10.py
unittest.main()
```

现在看看测试的输出。运行上面的命令，程序会输出以下结果。

```
.
-----
Ran 1 tests in 0.000s

OK
```

这个输出的意思是“运行了一个测试，结果 OK”。也就是说，测试通过了。如果测试过程中出现了一些问题，程序就会输出“`FAIL: test_forward(step10.SquareTest)`”，这表示测试失败了。

10.2 square 函数反向传播的测试

接下来添加针对 square 函数反向传播的测试。在刚刚实现的 SquareTest 类中添加以下代码。

steps/step10.py

```
class SquareTest(unittest.TestCase):
    ...

    def test_backward(self):
        x = Variable(np.array(3.0))
        y = square(x)
        y.backward()
        expected = np.array(6.0)
        self.assertEqual(x.grad, expected)
```

我们在代码中添加了一个名为 `test_backward` 的方法。在方法中通过 `y.backward()` 来求导，然后检查导数的值是否等于预期值。另外，代码中设置的预期值 (`expected`) `6.0` 是手动计算出来的。

现在再来测试一下上面的代码。输出结果如下所示。

```
..
-----
Ran 2 tests in 0.001s

OK
```

从结果来看，两项测试都通过了。之后可以采用和前面一样的做法来添加其他的测试用例（输入和预期值）。随着测试用例的增加，square 函数的可靠性也会增加。另外，我们也可以在修改代码时进行测试，以此来反复验证 square 函数的状态。

10.3 通过梯度检验来自动测试

我们在前面写了一个反向传播的测试方法。其中，导数的预期值是手动计算出来的。实际上有一种代替手动计算的自动测试方法，该方法叫作**梯度检验 (gradient checking)**。梯度检验是将数值微分得到的结果与反向传播得到的结果进行比较。如果二者相差很大，则说明反向传播的实现有问题。



步骤 4 实现了数值微分。数值微分容易实现，并能得出大体正确的导数值。因此，可以通过与数值微分的结果进行比较来测试反向传播的实现是否正确。

梯度检验是一种高效的测试方式，我们只要准备输入值即可。下面添加基于梯度检验的测试方法。这里我们会使用在**步骤 4**中实现的 `numerical_diff` 函数，这个函数的代码也会一并列出，兼作复习。

steps/step10.py

```
def numerical_diff(f, x, eps=1e-4):
    x0 = Variable(x.data - eps)
    x1 = Variable(x.data + eps)
    y0 = f(x0)
    y1 = f(x1)
    return (y1.data - y0.data) / (2 * eps)

class SquareTest(unittest.TestCase):
    ...

    def test_gradient_check(self):
        x = Variable(np.random.rand(1)) # 生成随机的输入值
        y = square(x)
        y.backward()
        num_grad = numerical_diff(square, x)
        flg = np.allclose(x.grad, num_grad)
        self.assertTrue(flg)
```

前面的代码会在进行梯度检验的 `test_gradient_check` 方法中生成一个随机的输入值，然后，通过反向传播求出导数，再利用 `numerical_diff` 函数通过数值微分求导，最后检查这两种方法得到的数值是否基本一致。这里使用的是 NumPy 函数 `np.allclose`。

`np.allclose(a, b)` 用于判断 ndarray 实例 `a` 和 `b` 的值是否接近。多近才算近是由 `np.allclose` 函数的参数 `rtol` 和 `atol` 指定的，指定方式如 `np.allclose(a, b, rtol=1e-05, atol=1e-08)` 所示。这时，如果 `a` 和 `b` 的所有元素满足以下条件，则返回 `True`。

$$|a - b| \leq (atol + rtol * |b|) \quad ①$$

另外，`atol` 和 `rtol` 的值有时需要根据要进行梯度检验的计算对象（函数）加以微调。调整基准可以参照参考文献 [5] 等资料。添加了上面的梯度检验之后，再次进行测试。这次得到的结果如下。

```
...  
-----  
Ran 3 tests in 0.001s  
  
OK
```

在可以自动微分的深度学习框架中，我们可以像上面那样利用梯度检验建立一个半自动的测试机制。这样可以系统地构建更广泛的测试用例。

10.4 测试小结

在创建 DeZero 方面，关于测试，了解以上知识就足够了。读者可以根据前面的步骤编写 DeZero 的测试代码。不过，本书之后的内容省略了测试相关的说明。如果读者觉得需要添加测试代码，请自行编写。

① `|·|` 表示绝对值。

通常，我们会在一个地方汇总管理所有测试代码文件。本书的测试代码也统一放在 `tests` 目录下（该目录下还包含了另行实现的用于测试的工具函数的代码）。有兴趣的读者可以看看这些测试代码，其中很多代码与我们在本步骤编写的代码类似。可以使用以下命令一起运行所有测试文件。

```
$ python -m unittest discover tests
```

像这样使用 `discover` 子命令后，`discover` 会在其后指定的目录下搜索测试文件，然后将所有找到的文件一起运行。在默认情况下，指定目录下符合 `test*.py` 模式的文件会被识别为测试文件（模式可以修改）。这样就能一次性运行 `tests` 目录下的所有测试文件了。



DeZero 的 `tests` 目录下也有以 Chainer 的结果为正确答案的测试。例如在测试 `sigmoid` 函数时，测试代码使用 DeZero 和 Chainer 分别对相同的输入进行计算，然后比较这两个输出是否大体相同。

DeZero 的 GitHub 仓库还集成了 Travis CI（参考文献 [9]）。Travis CI 是一个持续集成服务。在对 DeZero 的 GitHub 仓库^①的代码进行 `push` 和 `Pull Request` 操作时，测试会自动运行。如果结果有问题，Travis CI 会通过电子邮件通知。另外，DeZero 的 GitHub 仓库首页会显示图 10-1 这样的界面。



pypi v0.0.11

license MIT

build passing

图 10-1 DeZero 的 GitHub 仓库的首页界面

① 具体网址可通过 ituring.cn/article/521545 查看。

如图10-1所示，界面上会显示“build: passing”徽章。这个徽章意味着测试通过（如果测试失败，界面上会显示“build: failed”徽章）。通过与CI工具的协作，源代码能够不断得到测试。这样可以确保代码的可靠性。

DeZero现在还是一个小软件，我们会把它发展成更大的软件。引入本步骤介绍的测试机制后，我们有望持续保持代码的可靠性。以上就是第1阶段的内容。

★★★★★★

在第1阶段，我们一步一个脚印地创建了DeZero。最初的DeZero只有一个“小箱子”（变量），现在它已经发展到能够运行反向传播这种复杂算法的规模了。但是，现在实现的反向传播只能应用于简单的计算。从下一个阶段开始，我们将进一步扩展DeZero，使其可以应用于更复杂的计算。

专栏：自动微分

深度学习框架的核心技术是反向传播。有些资料将反向传播称为“自动微分”。需要注意的是，“自动微分”指代的是范围更加具体的一种技术，在学术领域尤其如此。下面是对自动微分的补充说明。



自动微分指的是自动求出导数的做法(技术)。“自动求出导数”是指由计算机(而非人)求出导数。具体来说，它是指在对某个计算(函数)编码后，计算机会自动求出该计算的导数的系统。

计算机程序求导的方法主要有3种。

第1种方法是数值微分。如同[步骤4](#)中实现的那样，首先给变量以微小的差异并执行普通计算(正向传播)，重复两次该操作，然后基于输出的差值计算得到近似的导数。数值微分虽然容易实现，但也存在一些问题，比如输出中包含误差、在处理多变量的函数时计算成本高等。

第2种方法是符号微分(symbolic differentiation)。这是使用导数公式求导的方法。输入是式子，输出也是式子(式子可以用树状结构的数据形式表示)。这种方法被用在 Mathematica 和 MATLAB 等软件中。



符号微分的输出是求导后的表达式，即导函数，这时还没有进行任何数值计算。在得到导函数之后，我们就可以求出某个特定值(如 $x = 3.0$)上的导数了。

符号微分的问题是式子很容易变得臃肿。特别是在不考虑优化的实现中，式子很快就会变“大”(可以说是式子“大爆炸”)。但深度学习中处理的计算需要高效地对大量变量求出导数“值”(而不是表达式)，这就需要使用更合适的方法了。

第3种方法是自动微分。这是一种采用链式法则求导的方法。我们对某个函数编码后，可以通过自动微分高效地求出高精度的导数。反向传播也是自动微分

的一种。更准确地说，自动微分可以大体分为两种：前向模式的自动微分和反向模式的自动微分。反向传播相当于反向模式的自动微分。



反向传播(反向模式的自动微分)将导数从输出到输入的方向传播。前向模式的自动微分则与之相反，导数的传播方向是从输入到输出。这两种方法都利用链式法则来求导，但路径并不相同。如果输出只有一个，要计算的是这个输出变量的导数，那使用反向模式的自动微分再合适不过了。许多机器学习问题的输出是一个变量，所以使用反向模式的自动微分。本书不对前向模式的自动微分做过多说明，对前向模式的自动微分感兴趣的读者可以阅读参考文献[6]和参考文献[7]。

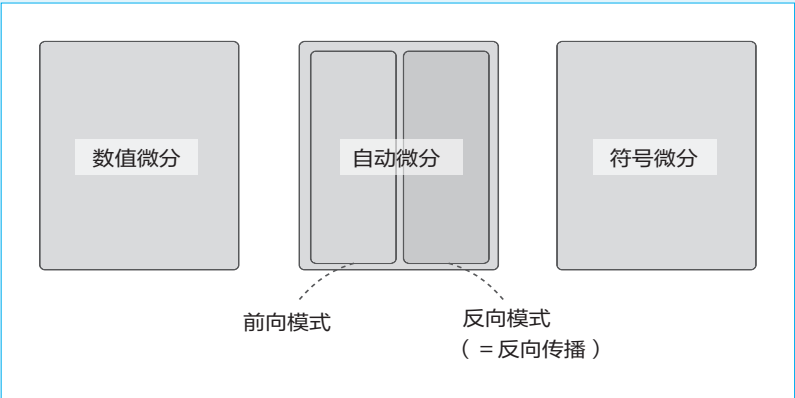


图 A-1 汇总了前面介绍的计算机程序求导的方法

如图 A-1 所示，自动微分是用计算机求导的一种方法。深度学习框架中实现的是反向模式的自动微分。不过有些资料不区分前向模式和反向模式，笼统地将反向传播称为“自动微分”。



在学术领域，自动微分的研究有很长的历史，积累了许多重要的研究成果。遗憾的是，此前自动微分与机器学习领域没有什么交集。近来，随着深度学习的蓬勃发展，自动微分领域受到越来越多的关注，机器学习和编程语言等领域与自动微分领域开始了新的交流。